

Space Invaders

Imran Kochi – ikochi1@aum.edu

Nigel Muchena –
nmuchena@aum.edu

Abstract—This electronic document is a “live” template and already defines the components of your paper [title, text, heads, etc.] in its style sheet. ***CRITICAL: Do Not Use Symbols, Special Characters, Footnotes, or Math in Paper Title or Abstract. (Abstract)**

Keywords—component, formatting, style, styling, insert (key words)

I. INTRODUCTION

The "Space Invaders" project is a 2D arcade-style shooting game developed using the Qt framework. Inspired by the classic gameplay mechanics of *Galaga* and the original *Space Invaders*, this project aims to recreate the fast-paced, retro gaming experience where players control a spaceship and defend Earth from waves of descending alien enemies. The game combines simple controls, dynamic enemy behavior, and increasing difficulty to keep players engaged. Built with Qt's powerful GUI and graphics tools, the project emphasizes modular design, smooth animations, and responsive input handling. This project not only serves as a tribute to vintage arcade games but also demonstrates the use of modern C++ and Qt techniques in game development.

II. EASE OF USE

Classic arcade games like *Space Invaders* and *Galaga* have had a lasting impact on game design, introducing fundamental mechanics such as wave-based enemies, simple controls, and progressively increasing difficulty. These elements have been widely studied and replicated due to their effectiveness in creating engaging gameplay. The success of these early games continues to influence modern recreations, with developers often using them as a foundation for learning or prototyping. Key design principles—such as predictable enemy patterns and escalating challenge—will be incorporated into this project to capture the original arcade feel.

From a technical standpoint, the Qt framework offers a practical environment for developing 2D games, especially for projects that prioritize cross-platform support and simplicity over complex 3D rendering. Although not a dedicated game engine, Qt provides essential tools like `QGraphicsScene` and `QTimer` that are well-suited for handling graphics, animation, and input in smaller-scale games. Previous projects recreating games like *Tetris* and *Pong* in Qt demonstrate its viability for similar retro-style games. This project leverages Qt's capabilities to build a modern take on *Space Invaders*, focusing on core gameplay mechanics while exploring features like sound integration and user interface design.

S. Kent, *The Ultimate History of Video Games: From Pong to Pokémon and Beyond*, New York, NY, USA: Three Rivers

Press, 2001. T. Iwatani, *Game Design Theory and Practice*, 2nd ed., Plano, TX, USA: Wordware Publishing, 2005.

The Qt Company, “Graphics View Framework,” Qt Documentation, [Online]. Available: <https://doc.qt.io/qt-6/qgraphicsview.html>

III. METHODOLOGY

A. Game Architecture

The *Space Invaders* project was built using a modular, object-oriented structure in C++. Qt's `QGraphicsScene`, `QGraphicsView`, and `QGraphicsItem` provide the foundation for rendering and controlling 2D objects. Every game entity—such as the player, enemies, bullets, and UI elements—is placed in its own class to ensure organization, reusability, and clarity.

B. Graphics and Animation

- All visual elements in the game are implemented as objects derived from `QGraphicsItem`. Animation and real-time behavior are coordinated through `QTimer`, which triggers periodic updates. This allows enemies to move smoothly across the screen, bullets to advance consistently, and other animated elements to behave as expected.

C. Input Handling

Keyboard inputs manage all player actions, including movement and shooting. Input is processed through the `QGraphicsView`, which handles event propagation within the scene. This ensures responsive controls and immediate feedback when the player interacts with the game.

D. Collision Detection

Collision detection uses Qt's built-in `collidingItems()` function. Player bullets destroy enemies on impact, enemies reaching the bottom of the screen trigger a game-over event, and enemy projectiles cause the player to take damage. These interactions form the core logic that drives gameplay outcomes.

E. Level Progression and Difficulty Scaling

Difficulty increases gradually as levels progress. Enemy speed, wave size, movement patterns, and attack frequency become more challenging with each new stage. Clearing a wave triggers the introduction of new patterns or faster enemy behavior, creating a progressively more intense gaming experience.

IV. ABSTRACT && CONCLUSION

A. Abstract

This project presents the design and implementation of a 2D *Space Invaders*-style game developed using C++ and the Qt framework. The primary objective was to recreate classic arcade

mechanics while applying core concepts of object-oriented programming, event-driven design, and graphical rendering. The game uses Qt's Graphics View Framework, modular class organization, timers for animation, collision detection, and scalable difficulty to provide a complete interactive experience. Throughout the project, emphasis was placed not only on building the game but also on strengthening essential programming skills such as abstraction, inheritance, memory management, and software architecture. The development process demonstrated Qt's capability for small-scale game creation and its usefulness as a learning tool for students transitioning from basic programming into GUI and interactive application development. This project ultimately showcases the successful recreation of a classic game and highlights meaningful growth in technical and problem-solving skills.

B. Conclusion

The Space Invaders project successfully achieved its goal of recreating the mechanics, difficulty progression, and feel of a classic arcade shooter using Qt and C++. The final game runs smoothly, features accurate collision detection, and provides a clear sense of escalating challenge through structured enemy waves. Beyond the functional results, this project served as an important milestone in our development as Computer Science students. We applied object-oriented programming in a practical environment, learned how to manage real-time graphical elements, and gained experience working with event-driven architecture. Implementing the game required problem-solving,

debugging, and designing reusable class structures—skills that are essential for future courses and professional software development. The project demonstrated that Qt is a strong educational tool and reinforced foundational concepts such as modularity, abstraction, and system design. Future improvements could expand the game with sound effects, additional enemy types, boss levels, and mobile platform support, but the current version already represents a complete and successful application of the concepts learned throughout the course.

REFERENCES

- [1] [1] S. Kent, *The Ultimate History of Video Games: From Pong to Pokémon and Beyond*. New York, NY, USA: Three Rivers Press, 2001.
- [2] T. Iwatani, *Game Design Theory and Practice*, 2nd ed. Plano, TX, USA: Wordware Publishing, 2005.
- [3] The Qt Company, "Graphics View Framework," Qt Documentation.
- [4] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," *Phil. Trans. Roy. Soc. London*, vol. A247, pp. 529–551, Apr. 1955.
- [5] J. Clerk Maxwell, *A Treatise on Electricity and Magnetism*, 3rd ed., vol. 2. Oxford: Clarendon, 1892.
- [6] I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in *Magnetism*, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963.
- [7] M. Young, *The Technical Writer's Handbook*. Mill Valley, CA: University Science, 1989.